

Jurnal etapa 1 – Petrenciuc Amelia-Andreea

1. Cerințe rezolvate

- S-a instalat Ollama și s-a utilizat un model de limbaj local (**llama3.2**) pentru a rula agentul.
- S-a implementat un server MCP (System File Manager MCP) care rulează pe portul 8002 și gestionează operații asupra unui director securizat (ADMIN_ROOT, create de mine pt testare în care am adaugat diverse fisere și directoare).
- Serverul expune următoarele cinci unelte (tools), îndeplinind și cerința "Bonus":
 - `list_directory(dir_path)`: Listează fișierele și directoarele (non-recursiv).
 - `get_file_content(file_path)`: Returnează conținutul unui fișier.
 - `list_files_recursive(dir_path)`: Listează recursiv *doar* fișierele dintr-un director.
 - `get_file_size(file_path)`: Returnează mărimea unui fișier specificat, în bytes.
 - `find_largest_file(dir_path)`: Caută recursiv și returnează cel mai mare fișier dintr-un director, împreună cu mărimea sa.
- S-a creat un agent (SysAdmin_Agent) care își încarcă instrucțiunile dintr-un fișier extern (instructions.txt).
- Agentul a fost testat folosind interfața web locală adk web (rulând pe portul 8000).

2. Modul de rezolvare

Pentru implementare, am utilizat pachetul **Ollama** pentru a rula local modelul **gemma3:4b** (specificat ca `ollama_chat/gemma3:4b`). Infrastructura de bază a proiectului se bazează pe biblioteca **google.adk**, care facilitează definirea agenților și comunicarea cu serverele MCP.

Am folosit **LiteLlm** ca un wrapper pentru modelul Ollama, permițând agentului să comunice eficient cu modelul de limbaj. Serverul MCP a fost implementat folosind **mcp.server.fastmcp**.

(`server_mcp.py`) rulează pe portul 8002 și expune cele cinci unelte menționate. Toate funcțiile de pe server sunt securizate pentru a opera exclusiv în directorul ADMIN_ROOT. Acest "sandbox" este implementat prin validarea căilor folosind `os.path.join`, `os.path.normpath` și verificarea `abs_path.startswith(ADMIN_ROOT)` pentru a preveni atacurile de tip "path traversal". Logica recursivă pentru `find_largest_file` a fost implementată ca o funcție internă (nested function) pentru a menține codul curat.

Agentul (`agent.py`) a fost configurat să fie flexibil, încărcându-și instrucțiunile dintr-un fișier extern `instructions.txt` folosind `Path(__file__).with_name("instructions.txt").read_text()`. Acest fișier conține prompt-ul detaliat care explică agentului AI cum să aleagă între cele cinci unelte disponibile. Agentul este expus printr-o interfață web locală folosind comanda `adk web`.

3. Probleme întâmpinate și modul de rezolvare

Pe parcursul implementării, au apărut mai multe provocări:

- Probleme de configurare a modelului LLM:
 - O primă dificultate a fost legată de comportamentul modelului `gemma3:4b`. Deși a fost instalat corect, modelul era reticent în a folosi uneltele definite. În loc să apeleze o funcție (de ex. `list_directory`), acesta tindea să halucineze că a

executat comanda, nu voia sa dea un raspuns. A fost nevoie de a schimba modelul cu llama3.2 .

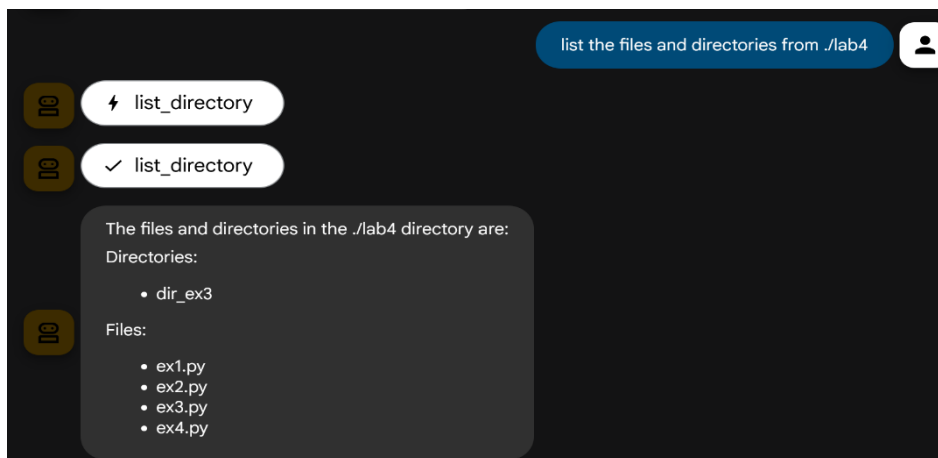
- Erori de import în VS Code:
 - O problemă neașteptată a apărut la scrierea codului în VS Code. Editorul nu părea să recunoască corect importurile, deși acestea erau instalate în mediul virtual (.venv).
 - Rezolvare: S-a descoperit că problema era cauzată de funcția de auto-completare (Tab). Soluția a fost să scriu manual fiecare linie de import (de ex. `from google.adk.agents import Agent`), literă cu literă, fără a accepta sugestiile editorului. Acest lucru a rezolvat erorile de import.
- Complexitatea "Prompt Engineering"-ului:
 - O altă dificultate majoră a fost scrierea fișierului `instructions.txt`. Deoarece agentul dispune de cinci unelte, dintre care unele par similare (ca `list_directory` și `list_files_recursive`), a fost nevoie ca instrucțiunile să fie extrem de explicite.
 - A fost necesar un efort considerabil pentru a detalia exact structura datelor returnate de fiecare funcție (de ex. dicționarul cu cheile `directories` și `files` vs. dicționarul cu cheia unică `files`) și pentru a stabili reguli clare care să ghideze agentul în alegerea uneltei corecte pentru fiecare scenariu.

4. Concluzii

În această fază a proiectului, am implementat cu succes un agent AI capabil să interacționeze într-un mod securizat și avansat cu sistemul de fișiere local. Am folosit un model LLM rulat local (gemma3:4b) și un server MCP personalizat cu cinci unelte distincte, incluzând funcționalități recursive.

Principalele lecții învățate au fost importanța critică a potrivirii tipurilor de date ("contractul") dintre serverul MCP și agent, tehnicile de depanare a problemelor de conexiune (Ollama, MCP) și necesitatea unui prompt engineering foarte detaliat pentru a ghida corect agentul în alegerea uneltelor.

TEST



list the file content from ./lab4/ex.1py

⚡ get_file_content

✓ get_file_content

The contents of the ./lab4/ex1.py file is:

```
import os
import sys

def find_largest_file(dirName):
    largest_file = ""
    max_size = 0

    for item_name in os.listdir(dirName):
        full_path = os.path.join(dirName, item_name)

        if os.path.isdir(full_path):
            sub_largest_file, sub_max_size = find_largest_file(full_path)
            if sub_max_size > max_size:
                max_size = sub_max_size
                largest_file = sub_largest_file

        elif os.path.isfile(full_path):
            size = os.path.getsize(full_path)
            if size > max_size:
```

list files recursive from ./lab4

⚡ list_files_recursive

✓ list_files_recursive

The files in the ./lab4 directory and its subdirectories are:

- lab4/ex1.py
- lab4/ex2.py
- lab4/ex3.py
- lab4/ex4.py
- lab4/dir_ex3/dir/test.txt

give me the largest file from ./lab4

⚡ find_largest_file

✓ find_largest_file

The largest file in the ./lab4 directory is ex3.py, and its size is 1313 bytes.

give me the size of ./lab4/ex2.py

⚡ get_file_size

✓ get_file_size

The size of the file ./lab4/ex2.py is 699 bytes.

